

Name: Muzammil Ahmed

ROLL NO: CT-24307

DSA

LAB 10:

Q1:

```
1  #include<iostream>
2  using namespace std;
3  class AVLTree{
4  private:
5  struct Node{
6  int key;
7  Node* left;
8  Node* right;
9  int height;
10 Node(int k) : key(k), left(nullptr), right(nullptr), height(1){}
11 };
12 Node* root;
13 int height(Node* node) {
14 return node ? node->height : 0;
15 }
16 int balanceFactor(Node* node) {
17 return node ? height(node->left) - height(node->right) : 0;
18 }
19 void updateHeight(Node* node) {
20 node->height = 1 + max(height(node->left), height(node->right));
21 }
22 Node* rotateRight(Node* y) {
23 Node* x=y->left;
24 Node* T2=x->right;
25 x->right=y;
26 y->left=T2;
27 updateHeight(y);
28 updateHeight(x);
29 return x;
30 }
31 Node* rotateLeft(Node* x) {
32 Node* y=x->right;
33 Node* T2=y->left;
34 y->left=x;
35 x->right=T2;
36 updateHeight(x);
37 updateHeight(y);
38 return y;
39 }
40 Node* balance(Node* node){
41 updateHeight(node);
42 int balance = balanceFactor(node);
43 if (balance > 1){
44 if (balanceFactor(node->left)<0)
45 node->left=rotateLeft(node->left);
46 return rotateRight(node);
47 }
48 // Right heavy
49 if (balance < -1){
50 if (balanceFactor(node->right)>0)
51 node->right=rotateRight(node->right);
52 return rotateLeft(node);
53 }
54 return node;
55 }
56 Node* minValueNode(Node* node){
57 Node* current=node;
58 while (current && current->left!=nullptr)
59 current = current->left;
60 return current;
61 }
```

```

62 Node* insert(Node* node, int key) {
63     if(!node){
64         return new Node(key);
65     }
66     if(key<node->key){
67         node->left=insert(node->left, key);
68     }
69     else if(key >node->key){
70         node->right = insert(node->right, key);
71     }
72     else{
73         return node;
74     }
75     return balance(node);
76 }
77 Node* deleteNode(Node* node, int key){
78     if (!node){
79         return node;
80     }
81     if(key<node->key){
82         node->left = deleteNode(node->left, key);
83     }
84     else if(key > node->key){
85         node->right=deleteNode(node->right, key);
86     }
87     else{
88         if(node->left == nullptr || node->right == nullptr) {
89             Node* temp =node->left ? node->left : node->right;
90             if(temp==nullptr){
91                 temp = node;
92                 node = nullptr;
93             }
94             else{
95                 *node = *temp;
96             }
97             delete temp;
98         }
99         else{
100             Node* temp=minValueNode(node->right);
101             node->key=temp->key;
102             node->right=deleteNode(node->right, temp->key);
103         }
104         if (node ==nullptr){
105             return node;
106         }
107         return balance(node);
108     }
109 void inOrder(Node* node){
110     if(!node)
111         return;
112     inOrder(node->left);
113     cout<<node->key << " ";
114     inOrder(node->right);
115 }
116 public:
117 AVLTree() : root(nullptr) {}
118 void insert(int key){
119     root=insert(root, key);
120 }
121 void deleteKey(int key){
122     root=deleteNode(root, key);
123 }
124 void inOrder(){
125     inOrder(root);
126     cout<< endl;
127 }

```

```

128     };
129     int main() {
130         AVLTree tree;
131         tree.insert(10);
132         tree.insert(20);
133         tree.insert(30);
134         tree.insert(20);
135         tree.insert(40);
136         tree.insert(50);
137         tree.insert(25);
138         cout << "In-order traversal before deletion: ";
139         tree.inOrder();
140         tree.deleteKey(20);
141         cout << "In-order traversal after deleting 20: ";
142         tree.inOrder();
143         return 0;
144     }

```

Output:

```

F:\3rd semistor\dsa\labs\lab' × + v
In-order traversal before deletion: 10 20 25 30 40 50
In-order traversal after deleting 20: 10 25 30 40 50

Process returned 0 (0x0)   execution time : 0.115 s
Press any key to continue.
|

```

Q2:

```
1  #include <iostream>
2  using namespace std;
3  class BST{
4  private:
5  struct TreeNode {
6  int value;
7  TreeNode* leftChild;
8  TreeNode* rightChild;
9  TreeNode(int v){
10 value=v;
11 leftChild=NULL;
12 rightChild=NULL;
13 };
14 TreeNode* root;
15 TreeNode* buildBalancedTree(int arr[],int startIndex, int endIndex){
16 if(startIndex>endIndex)
17 return NULL;
18 int middle=(startIndex+endIndex)/2;
19 TreeNode* newNode=new TreeNode(arr[middle]);
20 newNode->leftChild = buildBalancedTree(arr, startIndex, middle-1);
21 newNode->rightChild = buildBalancedTree(arr, middle+1, endIndex);
22 return newNode;
23 }
24 void printLevelOrder(TreeNode* node){
25 if(node==NULL)
26 return;
27 TreeNode* queue[100];
28 int front=0;
29 int back=0;
30 queue[back++]=node;
31 while(front<back){
32 TreeNode* current=queue[front++];
33 if (current!=NULL)
34 cout<<current->value<< " ";
35 else
36 {
37 cout <<"null ";
38 continue;
39 }
40 queue[back++]=current->leftChild;
41 queue[back++]=current->rightChild;
42 }
43 }
44 public:
45 BST(){
46 root=NULL;
47 }
48 void arrayToBST(int arr[], int size){
49 root=buildBalancedTree(arr, 0, size - 1);
50 }
51 void printTree(){
52 printLevelOrder(root);
53 cout << endl;
54 }
55 };
56 int main(){
57 int numbers[]={-10, -3, 0, 5, 9};
58 int size=5;
59 cout<<"Input: nums = [";
60 for(int i=0;i<size;i++) {
61 cout << numbers[i];
62 if(i<size-1)
63 cout << ", ";
64 }
65 cout<< "]"<<endl;
66 BST myTree;
67 myTree.arrayToBST(numbers,size);
68 cout << "Output: ";
69 myTree.printTree();
70 return 0;
71 }
```

Output:

```
"F:\3rd semistor\dsa\labs\lab' × + v
Input: nums = [-10, -3, 0, 5, 9]
Output: 0 -10 5 null -3 null 9 null null null null

Process returned 0 (0x0)   execution time : 0.130 s
Press any key to continue.
|
```

Q3

```
1  #include <iostream>
2  using namespace std;
3  class ListToBST{
4  private:
5  struct ListNode{
6  int val;
7  ListNode* next;
8  ListNode(int v){
9  val=v;
10 next=NULL;
11 }
12 };
13 struct TreeNode{
14 int val;
15 TreeNode* left;
16 TreeNode* right;
17 TreeNode(int v){
18 val=v;
19 left=NULL;
20 right=NULL;
21 }
22 };
23 ListNode* head;
24 TreeNode* root;
25 ListNode* getMid(ListNode* start, ListNode* end){
26 ListNode* slow = start;
27 ListNode* fast = start;
28 while (fast !=end && fast->next!=end){
29 slow=slow->next;
30 fast=fast->next->next;
31 }
32 return slow;
33 }
```

```

34  TreeNode* buildTree(ListNode* start, ListNode* end){
35      if(start==end)
36          return NULL;
37      ListNode* mid = getMid(start, end);
38      TreeNode* node=new TreeNode(mid->val);
39      node->left=buildTree(start, mid);
40      node->right=buildTree(mid->next, end);
41      return node;
42  }
43  void inorder(TreeNode* node){
44      if(node==NULL)
45          return;
46      inorder(node->left);
47      cout<<node->val << " ";
48      inorder(node->right);
49  }
50  public:
51  ListToBST(){
52      head=NULL;
53      root=NULL;
54  }
55  void insertList(int v){
56      if(head == NULL)
57          head=new ListNode(v);
58      else{
59          ListNode* temp=head;
60          while(temp->next !=NULL)
61              temp=temp->next;
62          temp->next=new ListNode(v);
63      }
64  }
65  void convert(){
66      root=buildTree(head, NULL);
67  }
68  void printInorder(){
69      inorder(root);
70      cout<<endl;
71  }
72  };
73  int main(){
74      ListToBST obj;
75      obj.insertList(60);
76      obj.insertList(70);
77      obj.insertList(80);
78      obj.insertList(90);
79      obj.insertList(100);
80      obj.convert();
81      cout << "Inorder Traversal of Height Balanced BST: ";
82      obj.printInorder();
83      return 0;
84  }

```

Output:

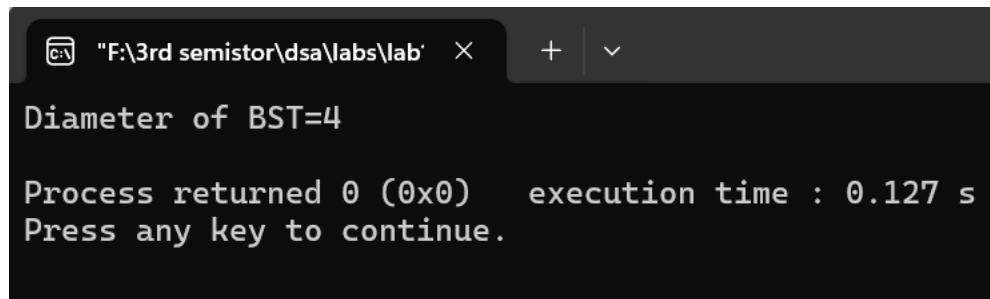
```
"F:\3rd semistor\dsa\labs\lab'  X + v
Inorder Traversal of Height Balanced BST: 60 70 80 90 100

Process returned 0 (0x0)    execution time : 0.129 s
Press any key to continue.
```

Q4:

```
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4  struct TreeNode{
5      int val;
6      TreeNode *left;
7      TreeNode *right;
8      TreeNode(int x):val(x),left(NULL),right(NULL){}
9  };
10 class Sol{
11     private:
12     int calculateHeightAndPath(TreeNode* node, int& longest_path) {
13     if(node == nullptr){
14         return 0;
15     }
16     int left_height=calculateHeightAndPath(node->left,longest_path);
17     int right_height=calculateHeightAndPath(node->right,longest_path);
18     int path_through_current= left_height+right_height;
19     longest_path =max(longest_path, path_through_current);
20     return 1 + max(left_height, right_height);
21     }
22     public:
23     int diameter(TreeNode* root){
24         int longest_path =0;
25         calculateHeightAndPath(root, longest_path);
26         return longest_path;
27     }
28 };
29 int main(){
30     TreeNode* root=new TreeNode(12);
31     root->left = new TreeNode(7);
32     root->right = new TreeNode(17);
33     root->left->left = new TreeNode(3);
34     root->left->right = new TreeNode(8);
35     root->left->left->left = new TreeNode(2);
36     root->left->right->right = new TreeNode(9);
37     Sol a;
38     cout<<"Diameter of BST="<< a.diameter(root)<<endl;
39     return 0;
40 }
```

Output:



```
"F:\3rd semistor\dsa\labs\lab' × + ∨  
Diameter of BST=4  
Process returned 0 (0x0)   execution time : 0.127 s  
Press any key to continue.
```